Mini Project Report

on

Playlist Builder: An Object Oriented Media Library

Submitted by

Project Members

Harsh Gidwani

Ansh Thakare

Darshan Nayak

Anushka Wani

SY B.Tech. (Artificial Intelligence & Data Science)

Course : Object Oriented Programming in CPP

Course Code : CSE1003B

Under the Internal Guidance of

Prof. Poonam Bhosle

Department of Computer Engineering and Technology
MIT World Peace University, Kothrud,
Pune 411 038, Maharashtra - India
2026-2027

DEPARTMENT OF COMPUTER ENGINEERING AND TECHNOLOGY

C E R T I F I C A T E

This is to certify that,

Harsh Gidwani

Ansh Thakare

Darshan Nayak

Anushka Wani

of SY-BTech. (Artificial Intelligence & Data Science) have completed their mini project titled "Playlist Builder: An Object Oriented Media Library" and have submitted this Object Oriented Programming in CPP Report for the academic year 2026-2027 (Sem-III).

Prof. Poonam Bhosle

Mini Project Guide

Department of CET

MIT World Peace University, Pune

Abstract

Playlist Builder is a C++17 mini project that stores songs, podcast episodes and audiobook chapters in one library, lets the user assemble playlists, and exports those playlists as standard M3U (.m3u) playlist files. The program is a small HTTP server with a browser user interface, so the same binary is both the object-oriented core and a usable desktop tool.

The domain model is built around an abstract base class `Media`. Three derived classes — `Song`, `PodcastEpisode` and `Audiobook` — add their own fields and override virtual functions for search, display and playlist export. Constructors validate every object before it exists. A hierarchy of exceptions derived from `AppException` reports empty titles, missing files, duplicate sources and empty playlists as JSON errors instead of crashing. The Standard Template Library supplies the library index (`unordered_map`), playlist order (`vector`), search and sort algorithms, and a template registry for playlists.

A seed library of twelve short WAV clips is generated on first launch so the player can be demonstrated immediately. Twenty-seven unit tests cover constructors, polymorphism, persistence and export. The browser interface follows a dark, glass-style music player layout with a now-playing disc, a searchable library, playlist editing and folder scanning.

List of Figures & Tables

SrNo	Title	Page No
1	Class diagram of Playlist Builder	5
2	Now playing and library user interface	10
3	Playlist list and mini player	10
4	Playlist detail with export actions	11
5	Add-media form (Song / Podcast / Audiobook)	11
6	Constructor exception shown in the browser	12
T1	List of classes and their roles	4
T2	Exception hierarchy and HTTP status codes	7
T3	Unit test cases and results	14
T4	Individual contributions	16

Contents

Abstract II

List of Figures & Tables III

Chapter No	Particulars	PageNo
1	Introduction	1
2	Problem Statement	2
3	Objectives	3
4	List of classes and class diagram	4
5	Description of OOP concepts used	6
6	Implementation Details and code	9
7	Test Cases and Results	14
8	Individual Contributions	16
9	Conclusion and Future Scope	17

Chapter 1
Introduction

Object Oriented Programming treats a program as a set of objects that hide their data and expose behaviour through well-defined operations. For a media library this is a natural fit: a song, a podcast episode and an audiobook chapter all have a title, a creator, a duration and a source file, yet each needs a different subtitle, a different search rule and a different line in a playlist file.

Playlist Builder was written for the Object Oriented Programming in CPP course (CSE1003B). The assignment required constructors, inheritance, polymorphism, the Standard Template Library and exception handling to appear as working parts of one program, not as isolated classroom examples. The program is therefore a complete tool: a C++ HTTP server that stores a library on disk, serves a browser player, and writes .m3u playlists that VLC or Apple Music can open.

1.1 Scope

The first release covers local files and HTTP(S) URLs, three media types, any number of playlists, folder scanning, search and sort, favourite flags, and JSON persistence. It does not stream commercial catalogues, does not require user accounts, and does not depend on third-party C++ libraries. JSON, HTTP and WAV generation are implemented in the project itself so every line that enforces a rule can be shown in this report.

1.2 Organisation of the report

Chapter 2 states the problem. Chapter 3 lists the objectives. Chapter 4 names every class and shows the class diagram. Chapter 5 maps each required OOP concept onto concrete code. Chapter 6 describes the implementation, the HTTP API and selected listings. Chapter 7 records the unit tests and the results of running them. Chapter 8 records individual contributions. Chapter 9 concludes and outlines future work.

Chapter 2
Problem Statement

People keep audio in mixed forms: MP3 albums, podcast feeds saved as files, and audiobook chapters. Consumer players treat these as a flat list of files. Course exercises that only print “Song created” never have to decide what happens when a title is blank, a file is missing, or two items share the same path.

The problem addressed by this mini project is: design and implement a type-safe media library in C++ in which songs, podcast episodes and audiobook chapters are related by inheritance, are stored and searched with the STL, cannot be constructed in an invalid state, and can be grouped into playlists that export a real .m3u file, with every failure reported through a typed exception that the user interface can display.

2.1 Why a web interface

A console menu would have been enough to call the classes. A browser player was chosen so the same objects can be seen, heard and exported. The interface is a pair of phone-sized panels (now playing and library) inspired by contemporary music-player design, with a purple gradient, a glass navigation bar and a circular progress disc. Polymorphism is visible in the type chip and subtitle of every row; exception handling is visible when the form reports `InvalidMediaException` instead of failing silently.

Chapter 3
Objectives

The mini project was planned against the following objectives.

1 To design an abstract `Media` base class and three derived types (`Song`, `PodcastEpisode`, `Audiobook`) that share identity, duration and source while keeping type-specific fields.

2 To validate every object in its constructor so an empty title, a negative duration, an unknown year or a missing episode number never becomes a live object.

3 To use virtual functions so a playlist of `shared_ptr<Media>` can export, search and display each item without a chain of type tests.

4 To store the library in STL containers (`unordered_map` for lookup, `vector` for order) and to search, sort, move and shuffle with standard algorithms.

5 To report user and storage failures through an exception hierarchy derived from `AppException`, mapped to HTTP status codes and shown in the UI.

6 To persist the library as JSON, seed a demo catalogue on first run, and export playlists as .m3u files that other players accept.

7 To present the library in a browser player that can play local WAV/MP3 files, build playlists and scan a folder of audio files.

8 To verify the design with automated unit tests covering constructors, polymorphism, persistence and export.

Chapter 4

List of classes and class diagram

4.1 List of classes

Class	Kind	Responsibility
Media	Abstract base	Shared identity, validation, JSON, M3U entry, live-object count.
Song	Derived	Album, genre, year; artist-title M3U label; album/genre search.
PodcastEpisode	Derived	Show name and episode number; episode-aware subtitle and search.
Audiobook	Derived	Narrator and chapter; chapter-aware subtitle and search.
MediaFactory	Factory	Builds the correct derived class from a JSON body.
Library	Container	Owns every Media item; O(1) lookup; filter, search and sort.
Playlist	Aggregate	Ordered shared tracks; move, shuffle, duration, M3U export.
Registry<T>	Class template	Id-keyed store used for playlists; next-id generation.
AppException	Exception base	Message plus HTTP status; virtual type() name.
InvalidMediaException	Exception	Constructor / field validation failures (HTTP 400).
BadRequestException	Exception	Malformed JSON or out-of-range playlist index (400).
NotFoundException	Exception	Unknown media, playlist or folder (404).
DuplicateException	Exception	Repeated id or source path (409).
EmptyPlaylistException	Exception	Export of a playlist with no tracks (422).
StorageException	Exception	Unreadable or unwritable save file (500).
Json	Value type	Minimal JSON parser and writer used for the API and save file.
Storage	Service	Load and save library.json atomically.
FolderScanner	Service	Walk a directory, parse “Artist - Title”, read WAV duration.
Seeder / SampleGenerator	Service	Create the demo library and synthesise short WAV clips.
PlaylistApp	Application	HTTP routes, mutex, persistence after every mutation.
Router / HttpServer	Infrastructure	Path matching, static files, Range requests, threads.

Table 1. Classes in Playlist Builder and the role each one plays.

4.2 Class diagram

Figure 1 shows inheritance from `Media` and `AppException`, aggregation of tracks inside a playlist, composition of the library and registry inside `PlaylistApp`, and the template parameter of `Registry<T>`.

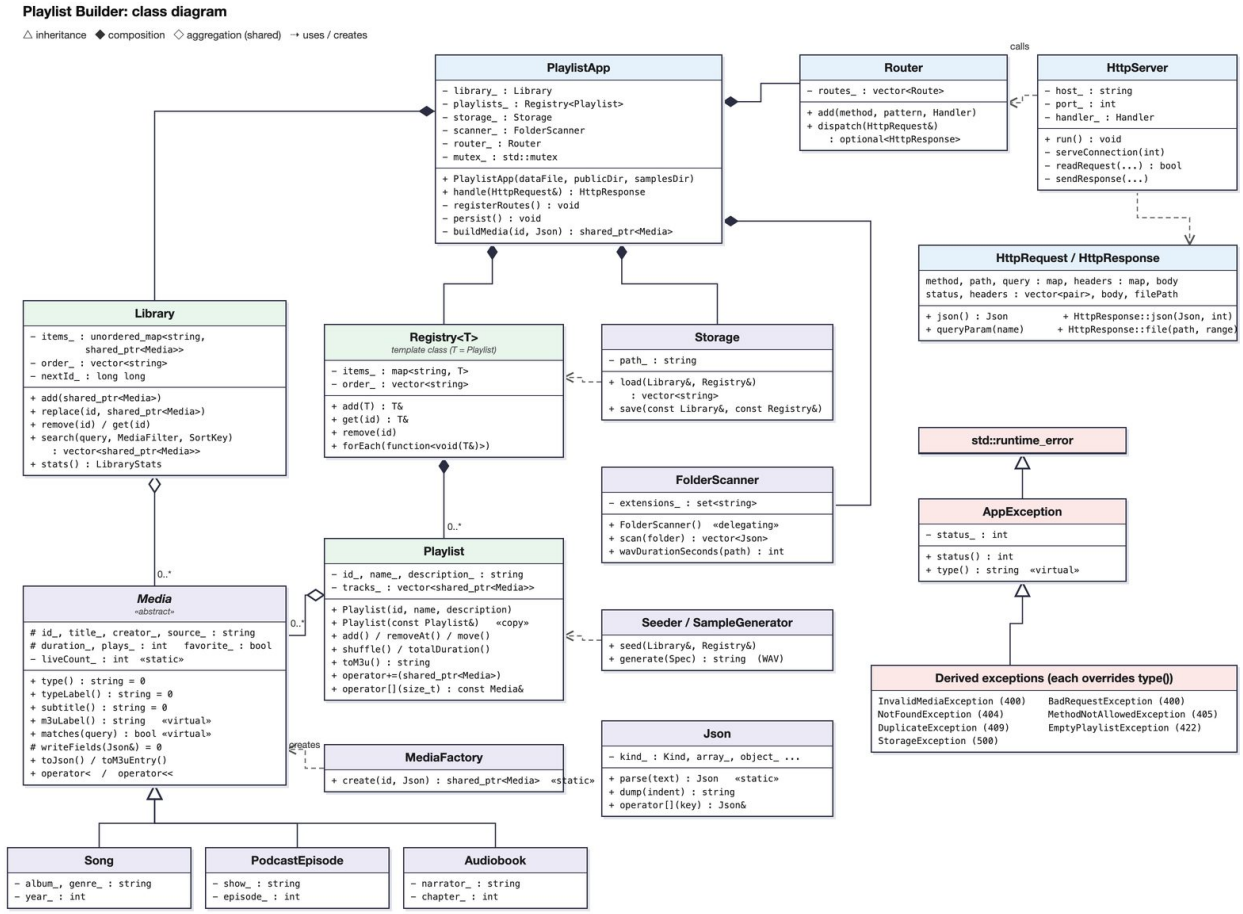


Figure 1. Class diagram of Playlist Builder.

Chapter 5
Description of OOP concepts used

5.1 Constructors

A constructor is the only path that creates a valid object. `Media::Media` trims and rejects empty titles, creators and sources, and rejects negative durations. `Song` additionally checks that a non-zero year lies in 1900–2100. `PodcastEpisode` and `Audiobook` require episode and chapter numbers of at least 1. `Playlist` rejects a blank name. Because these checks throw `InvalidMediaException` (or `BadRequestException`) before the object is usable, the rest of the program never has to test “is this title empty?”.

`Playlist` also has an explicit copy constructor, used by the Duplicate action: the copy shares the same `Media` objects (aggregation) but has its own name and track vector. `FolderScanner` uses a delegating constructor for its default audio-extension set. A static live-object counter is incremented in every `Media` constructor and decremented in the virtual destructor, which is how the tests prove that derived objects are destroyed through a base pointer.

```
Media::Media(std::string id, std::string title, std::string creator,
             int durationSeconds, std::string source)
    : id_(std::move(id)),
      title_(requireText(title, "Title")),
      creator_(requireText(creator, "Creator")),
      source_(requireText(source, "Source", 2048)) {
    if (id_.empty()) throw InvalidMediaException("Media id cannot be empty");
    setDuration(durationSeconds);
    ++liveCount_;
}
```

5.2 Inheritance

Inheritance expresses “is-a”. A song is a media item, so `Song` publicly derives from `Media` and reuses id, title, creator, duration, source, favourite and play-count. Only album, genre and year live on the derived class. The same pattern is used for podcast episodes (show, episode number) and audiobooks (narrator, chapter). Protected helpers `requireText` and `optionalText` are inherited so every subclass validates strings the same way.

A second hierarchy sits under `std::runtime_error`: `AppException` adds an HTTP status, and seven concrete types override `type()`. One `catch (const AppException&)` in `PlaylistApp::handle` is enough to turn any of them into a JSON error body.

5.3 Polymorphism

Polymorphism is the reason a playlist can hold mixed types. `Media` declares pure virtual `type()`, `typeLabel()`, `subtitle()` and `writeFields()`, and overridable

`m3uLabel()` and `matches()`. `Playlist::toM3u` walks `vector<shared_ptr<Media>>` and calls `toM3uEntry()` on the base; each derived class supplies its own label, so a song is written as “Roary - Odyssey” and a podcast as “Compile Time - Ep. 12: Why Virtual Functions Matter” without a `dynamic_cast` in the exporter.

`MediaFactory::create` is the other polymorphic seam: given a JSON object it returns a `shared_ptr<Media>` to the right subclass, or throws if the type is unknown. Operators are overloaded on the domain types: `Playlist::operator+=` appends a track, `operator[]` returns a const Media reference, `Media::operator<` orders by title, and `operator<<` prints a readable line for debugging and tests.

```
std::string Playlist::toM3u() const {
    if (tracks_.empty()) throw EmptyPlaylistException(name_);
    std::string out = "#EXTM3U\n#PLAYLIST:" + name_ + "\n";
    for (const auto& media : tracks_) out += media->toM3uEntry();
    return out;
}
```

5.4 Standard Template Library

The STL is used as the default data layer, not as a demonstration of one container. The library keeps an `unordered_map<string, shared_ptr<Media>>` for O(1) lookup and a `vector<string>` for insertion order. Playlists are a `vector` of shared pointers so the same song can appear twice. `Registry<T>` is a class template parameterised on the stored type; it is instantiated as `Registry<Playlist>`.

Algorithms do the rest: `copy_if` and a predicate implement search and type filters; `stable_sort` implements the five sort keys; `rotate` implements drag-to-reorder; `shuffle` randomises a playlist; `accumulate` totals duration; `remove_if` / `replace_if` update tracks when media is deleted or edited; `any_of` detects a duplicate source; `transform` builds the ordered snapshot; `mismatch` prevents path traversal when serving static files. `std::filesystem` walks folders. `std::thread` and `std::mutex` run the server.

5.5 Exception handling

Every user-visible failure is an object. The hierarchy and the status each type maps to are shown in Table 2. `PlaylistApp::handle` catches `AppException` first (typed JSON error) and `std::exception` second (unexpected 500). The browser prints the exception type next to the message, so a failed add is labelled `InvalidMediaException` rather than a generic “error”.

Exception	HTTP	Typical cause
InvalidMediaException	400	Empty title, bad year, missing file, unknown type.
BadRequestException	400	Malformed JSON, missing field, index out of range.

Exception	HTTP	Typical cause
NotFoundException	404	Unknown media id, playlist id, or scan folder.
MethodNotAllowedException	405	POST/PUT/DELETE on a static page.
DuplicateException	409	Same id or same source path already in the library.
EmptyPlaylistException	422	Export requested on a playlist with zero tracks.
StorageException	500	library.json cannot be read or written.

Table 2. Exception hierarchy and the HTTP status each type reports.

Chapter 6

Implementation Details and code

6.1 Language, tools and layout

The server is C++17, compiled with `c++ -std=c++17 -O2 -Wall -Wextra -Wpedantic` and linked with pthreads. There is no CMake dependency and no external package. A Makefile produces `bin/playlist-builder` and `bin/tests`. Headers live in `include/`, implementations in `src/`, the player in `public/`, and tests in `tests/test_core.cpp`. A Dockerfile builds the same binary on Debian for optional deployment.

6.2 Runtime architecture

On start, `main` parses `--port`, `--host`, `--data`, `--public` and `--samples`. `PlaylistApp` either loads `data/library.json` or, on first run, asks `Seeder` to synthesise twelve WAV clips and two playlists. `HttpServer` accepts connections on a thread each, parses the request, and calls `PlaylistApp::handle`. Mutations take a mutex and call `Storage::save`, which writes a temporary file and renames it so a crash cannot leave a truncated JSON document.

Local audio is served from `GET /stream/:id` with HTTP Range support so the browser can seek. Remote items keep their original URL. Playlist export is `GET /api/playlists/:id/export.m3u` with a `Content-Disposition` filename derived from the playlist name.

6.3 User interface

The page is a pair of 424-pixel panels on desktop and a single full-screen panel on a phone. The left panel is now playing: title, heart, circular glass disc with a progress arc, transport controls and a glass navigation bar. The right panel is the library, playlists, add-media form or activity log, with a mini player when something is playing. Album art is a deterministic SVG generated from the item id, so the demo does not need image files. Figures 2 to 6 are screenshots of the running server at `http://127.0.0.1:8080`.



Figure 2. Now playing (Skin) beside the library list.

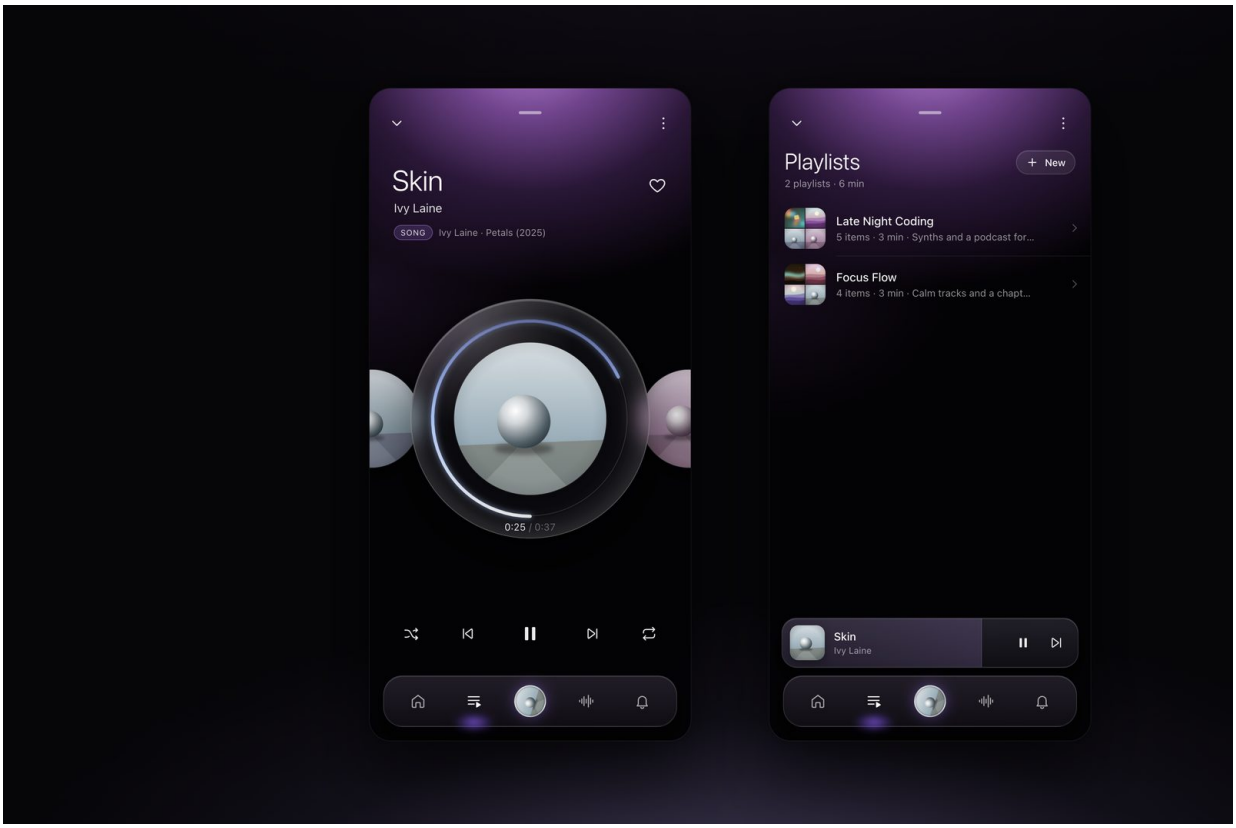


Figure 3. Playlist list with the mini player and progress fill.



Figure 4. Playlist detail: play, shuffle, export .m3u, duplicate, rename, delete.

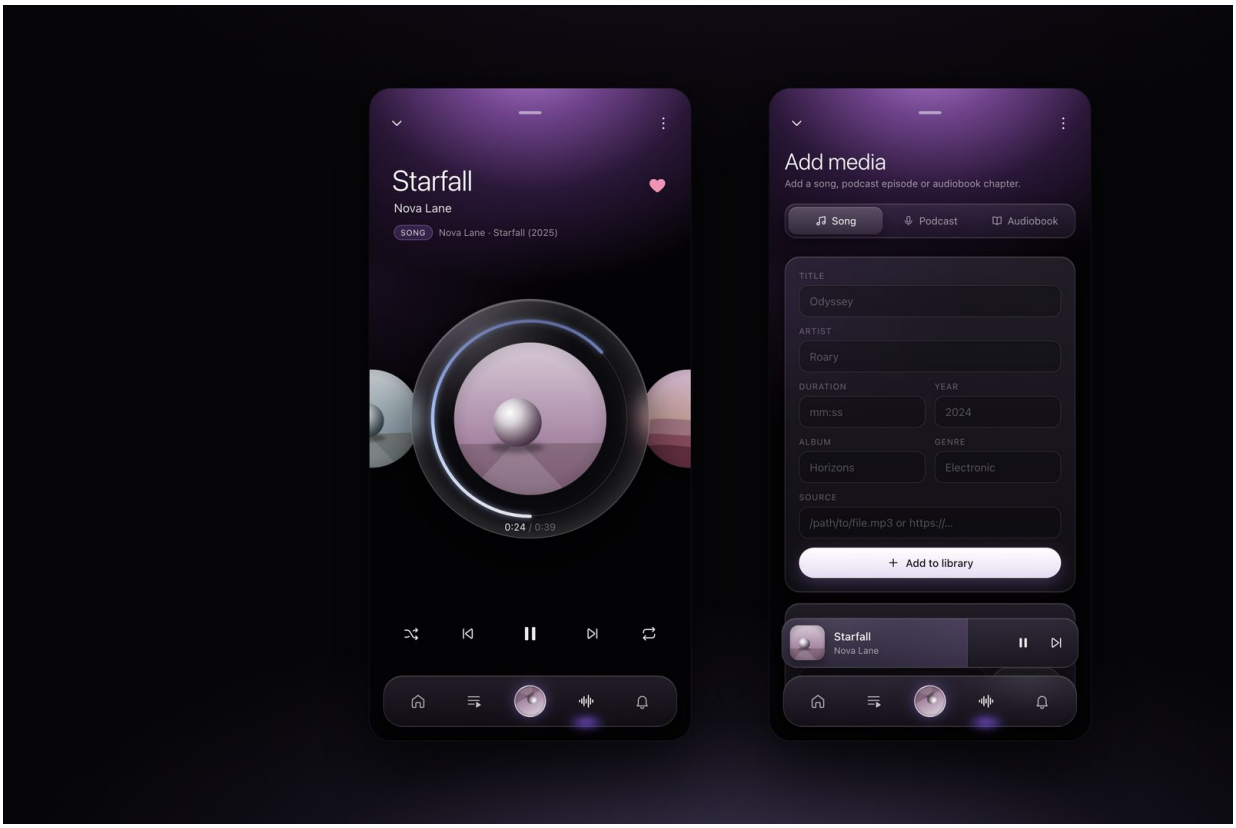


Figure 5. Add-media form. Extra fields change with Song, Podcast or Audiobook.

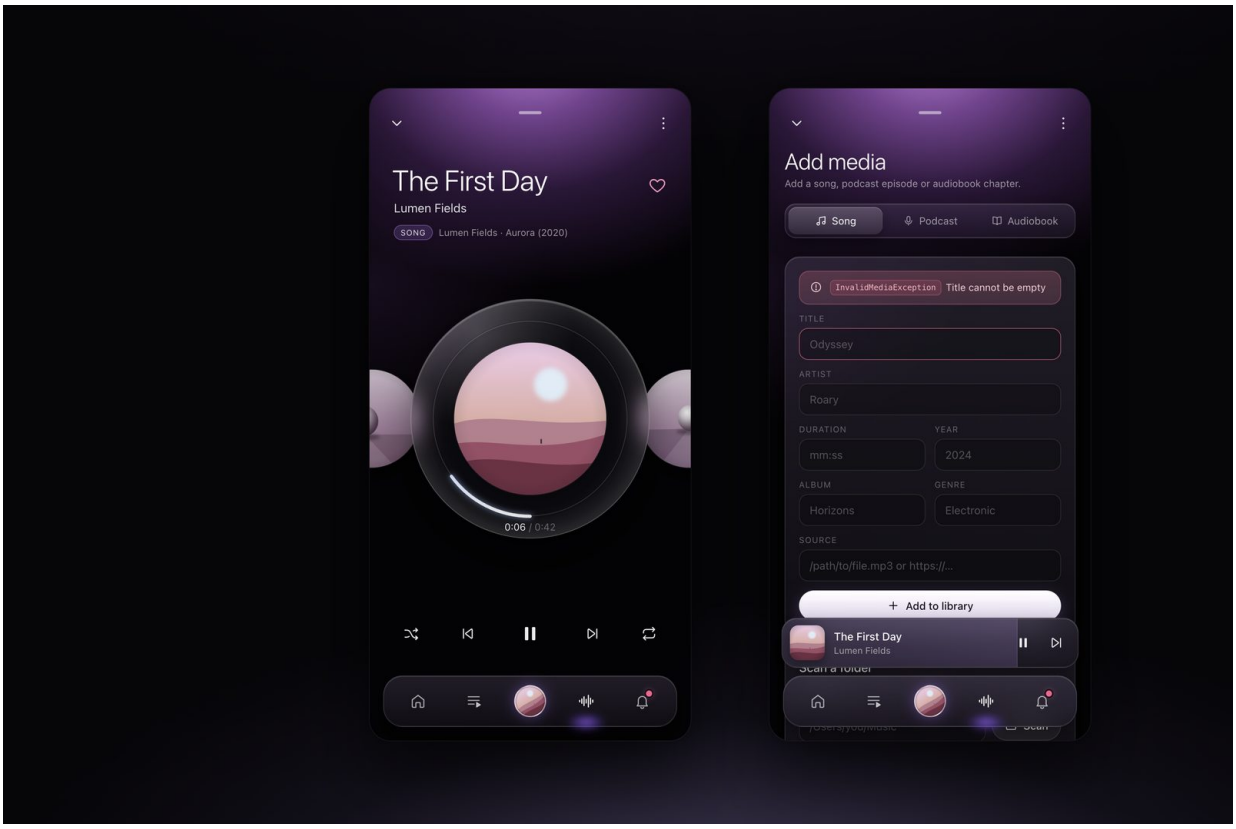


Figure 6. Submitting an empty title returns InvalidMediaException in the form.

6.4 Selected implementation

Derived constructors call the base constructor, then validate their own fields. Virtual `subtitle()` is what the now-playing chip and the library row both display:

```
std::string Song::subtitle() const {
    std::string text = creator();
    if (!album_.empty()) text += " · " + album_;
    if (year_ != 0) text += " (" + std::to_string(year_) + ")";
    return text;
}

std::string PodcastEpisode::subtitle() const {
    return show_ + " · Episode " + std::to_string(episode_);
}

std::string Audiobook::subtitle() const {
    std::string text = creator() + " · Chapter " + std::to_string(chapter_);
    if (!narrator_.empty()) text += " · read by " + narrator_;
    return text;
}
```

Library search is a filter plus a virtual match, then a stable sort. The type of each object is not tested with `if (song)` except when the user asked for a type chip; matching itself is polymorphic:

```
std::vector<std::shared_ptr<Media>> Library::search(
        const std::string& query, MediaFilter filter, SortKey sort) const {
    const std::string needle = toLower(query);
    auto keep = [&](const std::shared_ptr<Media>& m) {
        switch (filter) {
            case MediaFilter::Songs:      if (m->type() != "song") return false; break;
            case MediaFilter::Podcasts:   if (m->type() != "podcast") return false; break;
```

```
            case MediaFilter::Audiobooks: if (m->type() != "audiobook") return false; break;
            case MediaFilter::Favorites:  if (!m->favorite()) return false; break;
            case MediaFilter::All: break;
        }
        return m->matches(needle);
    };
    // copy_if into a vector, then stable_sort by the chosen key
    ...
}
```

The web layer does not invent its own error strings. It rethrows nothing: it catches the domain exception and copies `type()`, `what()` and `status()` into JSON.

```
} catch (const AppException& e) {
    return errorResponse(e.type(), e.what(), e.status());
}
```

How to run the project from the folder `sem3-code/oop/playlist-builder`:

```
make            # builds bin/playlist-builder
make test       # 27 unit tests
make run        # http://127.0.0.1:8080
```

Chapter 7

Test Cases and Results

Automated tests live in `tests/test_core.cpp` and are built with `make test`. They construct real domain objects, including short WAV files written by `SampleGenerator`, and they assert both the happy path and the exception type. All 27 cases passed on Apple clang 21 (macOS) as recorded below.

ID	Test case	Expected	Result
TC01	Song constructor stores trimmed title and year	title Odyssey, year 2024	PASS
TC02	Empty title rejected	InvalidMediaException	PASS
TC03	Negative duration rejected	InvalidMediaException	PASS
TC04	Year 1800 rejected	InvalidMediaException	PASS
TC05	Podcast episode 0 rejected	InvalidMediaException	PASS
TC06	Virtual subtitle() per derived class	three different strings	PASS
TC07	MediaFactory builds Audiobook	dynamic_cast succeeds	PASS
TC08	Unknown type rejected	InvalidMediaException	PASS
TC09	Search matches Song album	one hit	PASS
TC10	Filter songs, sort by title	alpha then Zeta	PASS
TC11	Duplicate id and source rejected	DuplicateException	PASS
TC12	Missing id	NotFoundException	PASS
TC13	Playlist +=, duration, operator[]	size 2, 150 s	PASS
TC14	move() uses std::rotate	new order	PASS
TC15	Out-of-range move	BadRequestException	PASS
TC16	Empty playlist export	EmptyPlaylistException	PASS
TC17	M3U uses each type’s label	EXTINF song and podcast lines	PASS
TC18	Copy constructor independent	original size unchanged	PASS
TC19	removeMedia drops every occurrence	two removed	PASS
TC20	Blank playlist name	BadRequestException	PASS
TC21	JSON round-trip and bad input	parse / BadRequestException	PASS
TC22	Storage save/load	library and playlist restored	PASS
TC23	Corrupt save file	StorageException	PASS

ID	Test case	Expected	Result
TC24	Folder scanner Artist - Title + WAV duration	Nova Lane / Starfall / 3 s	PASS
TC25	Missing scan folder	NotFoundException	PASS
TC26	Virtual destructor via base pointer	liveCount returns to baseline	PASS
TC27	operator<< printable line	[Song] Skin - … (2:05)	PASS

Table 3. Unit test cases and results (`make test`: 27 tests, 0 failed).

7.1 Manual checks on the running server

With `bin/playlist-builder` listening on port 8080 the following browser and HTTP checks were performed:

- Library lists 12 items (8 songs, 2 podcasts, 2 audiobook chapters) and 3 favourites.
- Clicking Skin starts playback, updates the document title, lights the progress arc and shows the mini player.
- Playlists Late Night Coding and Focus Flow open, shuffle, and export `Late-Night-Coding.m3u` with EXTINF lines that differ by type.
- `POST /api/media` with an empty title returns HTTP 400 and `{"type":"InvalidMediaException","message":"Title cannot be empty"}`; the add form shows the same badge (Figure 6).
- `GET /api/playlists/p3/export.m3u` on a newly created empty playlist returns HTTP 422 `EmptyPlaylistException`.
- Search, type chips, favourite heart, next/previous and folder-scan path validation all respond without a crash.

Chapter 8
Individual Contributions

This mini project was completed jointly by Harsh Gidwani, Ansh Thakare, Darshan Nayak and Anushka Wani, under the internal guidance of Prof. Poonam Bhosle. Table 4 lists each member.

SrNo	Project member	Contribution
1	Harsh Gidwani	Design, C++ implementation, browser UI, testing and this report (joint).
2	Ansh Thakare	Design, C++ implementation, browser UI, testing and this report (joint).
3	Darshan Nayak	Design, C++ implementation, browser UI, testing and this report (joint).
4	Anushka Wani	Design, C++ implementation, browser UI, testing and this report (joint).

Table 4. Individual contributions.

The four members shared the class design, the exception cases, the user interface and the final demonstration.

Chapter 9
Conclusion and Future Scope

9.1 Conclusion

Playlist Builder shows that the five OOP topics required by the course can sit inside one program that someone would actually open. Constructors refuse bad data. Inheritance shares the fields that songs, podcasts and audiobooks have in common. Polymorphism writes the right playlist line and the right subtitle through a base pointer. The STL holds, searches and reorders those objects. Exceptions turn every failure into a message the browser can show. Twenty-seven automated tests and a working player at `http://127.0.0.1:8080` support those claims.

9.2 Future scope

1. Read ID3 / Vorbis tags so scanned files pick up album art and duration without a WAV-only parser.
2. Persist playlists as separate .m3u files that the OS file manager can open, in addition to the JSON save.
3. Add a `Video` derived class if the course extends the hierarchy later.
4. Replace the in-process JSON file with SQLite while keeping the same Media interface.
5. Package a signed macOS build (or a Docker image on a free host) so the player can be opened from a bookmark without compiling.

— End of report —